BHASKARACHARYA NATIONAL INSTITUTE FOR SPACE APPLICATIONS AND GEO-INFORMATICS

WEEKLY PROGRESS REPORT (25/05/2026 – 31/05/2026)

WEEK 2

Project Name	OSINT Collection from Social Media Network

PROJECT DESCRIPTION :	Continuation of Week 1 OSINT pipeline. Week 2 focused on PostgreSQL database integration, tweet-free Neo4j graph schema migration, cross-run scrape deduplication, master tweet CSV export, FastAPI backend development, and building the complete linkmap OSINT Intelligence Dashboard frontend with 7 pages.
GROUP MEMBER :	Rishav Pal, Sayantan Mandal, Shivam Kumar
GROUP ID :	2026G268
GROUP GUIDE :	Dr. JHUMMARWALA ABDUL
GROUP COORDINATOR :	Dr. JHUMMARWALA ABDUL
COLLEGE GUIDE NAME :	Dr. SHWETA SAHARAN
COLLEGE GUIDE No. :	9509676076
COLLEGE GUIDE EMAIL. :	SHWETA.SAHARAN@GSV.AC.IN
COLLEGE NAME :	GATI SHAKTI VISHWAVIDYALAYA

WEEKLY PROGRESS REPORT (25/05/2026 – 31/05/2026)

WEEK 2 — DAILY WORK LOG

Day 1–2 | 25–26 May 2026 | PostgreSQL Integration & Data Pipeline Enhancement

The first two days of Week 2 were dedicated to integrating PostgreSQL as the primary relational data store for raw tweet data, enabling structured querying and analysis separate from the Neo4j graph database. The team also added cross-run tweet deduplication to avoid re-scraping already collected tweets.

- Designed and implemented storage/postgres_writer.py — a PostgreSQL persistence layer using psycopg2. Defined two tables: profiles (handle, display_name, bio, followers_count, following_count, verified, profile_url, scraped_at) and tweets (tweet_id, author_handle, text, timestamp, likes, retweets, replies, quote_count, is_reply, is_retweet, is_quote, hashtags TEXT[], mentioned_handles TEXT[], links JSONB, topics TEXT[], scraped_at). Uses ON CONFLICT DO UPDATE for idempotent upserts.
- Implemented storage/tweet_csv_writer.py — exports all scraped tweets to a single master CSV file (data/tweets/all_tweets.csv). Uses pipe (|) separator for array fields (hashtags, mentions, topics, links) to avoid conflicts with commas in tweet text. Atomic append-mode writes with QUOTE_ALL for safe newline/comma handling.
- Added cross-run deduplication to scraper/tweet_scraper.py — new seen_ids parameter loads already-scraped tweet IDs from both the existing JSONL file and the master CSV before starting each handle. The scraper skips any tweet whose ID already exists, preventing duplicate collection on re-runs.
- Added load_seen_ids_from_jsonl() static method to TweetScraper — fast-reads only the tweet_id field from JSONL files using ujson, skipping the profile line. Returns a set for O(1) lookup.
- Updated main.py to wire deduplication — before each handle scrape, merges IDs from CSV (global_seen_ids) and JSONL (jsonl_ids) into combined_seen and passes to TweetScraper.
- Created scratch/verify_csv.py — validation script that checks for duplicate tweet IDs, verifies JSONL-to-CSV row count consistency, and prints a PASS/FAIL summary.
- Ran storage/backfill_postgres.py to migrate all existing parsed data from merged.json into the PostgreSQL database. Verified 1,601 tweets successfully inserted.

Day 3 | 27 May 2026 | Neo4j Tweet-Free Graph Schema Migration

Day 3 focused on migrating the Neo4j graph to a leaner tweet-free schema, removing Tweet nodes entirely and replacing tweet-intermediary relationships with direct entity-to-entity edges.

- Rebuilt graph/queries/load_nodes.cypher — removed Tweet node loading entirely. Now loads only User, Hashtag, Link, and Topic nodes.
- Rebuilt graph/queries/load_relationships.cypher — removed POSTED, MENTIONED_IN, HAS_HASHTAG (tweet-based), CONTAINS_LINK, REPLY_TO, RETWEET_OF, QUOTES, TAGGED_WITH. Added three new direct relationships: USED_HASHTAG (User→Hashtag with count), SHARED_LINK (User→Link with count), SHARES_HASHTAG_WITH (User→User via shared hashtag).
- Updated graph/transformer.py — added _build_tweet_free_rels() method that derives the three new relationship CSVs by collapsing through tweet data without creating Tweet nodes. Generates: rels_user_hashtag.csv, rels_user_link.csv, rels_user_shared_hashtag.csv.
- Updated graph/queries/constraints.cypher — removed tweet_id uniqueness constraint and tweet_timestamp index. Kept all User, Hashtag, Link, Topic constraints.
- Rebuilt graph/queries/campaign_detection.cypher — rewrote all 5 queries to operate on the new tweet-free schema using USED_HASHTAG, SHARES_HASHTAG_WITH, AUTHORED_ABOUT, and SHARED_LINK edges.
- Ran MATCH (n) DETACH DELETE n in Neo4j Browser to wipe old graph, then ran python main.py --mode transform && python main.py --mode load to rebuild cleanly.
- Verified new graph: 469 User nodes, 303 Hashtag nodes, 425 Link nodes, 12 Topic nodes. Relationships: USED_HASHTAG (288), SHARED_LINK (0 — link data pending fix), SHARES_HASHTAG_WITH (35), AUTHORED_ABOUT (93), MENTIONED_BY_USER (437), BELONGS_TO (267).

WEEK 2 — DAILY WORK LOG (continued)

Day 4 | 28 May 2026 | Backend Architecture Restructure & FastAPI Development

Day 4 restructured the project to properly separate backend Python code from frontend static files, and built the complete FastAPI backend with all API routes.

- Restructured project: moved all Python API code from frontend/api/ into a dedicated backend/ folder. frontend/ now contains only static HTML/CSS/JS files. Updated all import paths (sys.path parents depth changed from 3-4 levels to 1-2 levels).
- Created backend/postgres_client.py — dedicated PostgreSQL client using psycopg2 ThreadedConnectionPool (minconn=2, maxconn=10). Provides execute_query() returning list of dicts via RealDictCursor, execute_write() for INSERT/UPDATE/DELETE, and verify_connection() for health checks.
- Built backend/routes/dashboard.py — serves KPI stats combining Neo4j node counts with PostgreSQL tweet counts. Fetches top hashtags and topic distribution from Postgres unnested arrays.
- Built backend/routes/handles.py — Neo4j for graph metrics (hashtag_count, link_count, mention_count), enriched with PostgreSQL tweet_count per handle. Full pagination, search, sort.
- Built backend/routes/hashtags.py — frequency data from PostgreSQL unnested hashtag arrays with CTE pattern. Co-occurrence data from Neo4j SHARES_HASHTAG_WITH edges. Timeline from PostgreSQL DATE_TRUNC grouping.
- Built backend/routes/topics.py, campaigns.py, influence.py, links.py, graph.py — all routes wired to appropriate data sources (Postgres for tweet data, Neo4j for graph relationships).
- Updated backend/server.py /health endpoint to check both Neo4j and PostgreSQL connections, returning {"neo4j": "Online/Offline", "postgres": "Online/Offline"}.
- Added security: parameterised queries everywhere (no f-string SQL), input validation via Pydantic, whitelist-based sort_by enum, rate limiting (100 req/min), CORS, security headers middleware.

Day 5–6 | 29–30 May 2026 | Frontend Dashboard Development

Days 5 and 6 were dedicated to building the complete linkmap OSINT Intelligence Dashboard — a 7-page professional web application with editorial Swiss design aesthetic, connecting to both the FastAPI backend and Neo4j graph.

- Established global design system: off-white background (#F0EDE8), pure black typography, Playfair Display for display headings (96px thin), JetBrains Mono for all data/numbers, Inter for body text. Zero border-radius, no shadows, 1px black dividers only.
- Built frontend/index.html (Overview) — animated display heading with sequential word reveal, live stat strip fetching from /api/dashboard/stats, abstract SVG network animation with traveling dot particles on edges, project description and metrics.
- Built frontend/handles.html — Handle Intelligence page with cluster frequency bar chart (animated left-to-right on load), paginated sortable table with search, filter bar (search handle, min followers, cluster, sort by).
- Built frontend/network.html — Influence Network page with D3.js force-directed graph fetching from Neo4j via /api/graph/full. Dark navy canvas with Neo4j-style coloured nodes (purple=User, green=Hashtag, orange=Link, blue=Topic). Click node → Node Inspector panel. Zoom/pan, show labels toggle, SVG export.
- Built frontend/hashtags.html — Hashtag Analysis page with top-10 frequency bar chart, co-occurrence table, burst campaign signal display, hashtag timeline chart.
- Built frontend/topics.html — Topic Clusters page with expandable topic rows, sidebar vertical bar distribution chart, filter bar (search topic, min tweets, min handles).
- Built frontend/campaigns.html — Campaign Detection page with coordinated activity alert banner, burst events table, hashtag cluster radial hub SVG, campaign timeline.
- Implemented skeleton loaders across all pages — shimmer animation in same off-white palette while API data loads. All JS extracted to separate module files in frontend/js/.

Day 7 | 31 May 2026 | Tweet Analysis Page, Filters & Integration Testing

The final day added the Tweet Analysis page — a full PostgreSQL-powered tweet search and filter interface — and completed integration testing of all pages against live data.

- Built backend/routes/tweet_analysis.py — three endpoints: GET /tweet-analysis/tweets (filterable paginated tweet table), GET /tweet-analysis/stats (aggregate statistics), GET /tweet-analysis/filter-options (dropdown values). All filters use parameterised SQL only.

- Filters implemented: handle (exact match), topic (ANY(topics) array match), hashtag (ANY(hashtags) array match), keyword (ILIKE full-text search in tweet text), date_from / date_to (timestamptz range), is_reply (bool), is_retweet (bool).
- Built frontend/tweet-analysis.html and frontend/js/tweet-analysis.js — full tweet analysis page with filter bar (all 6 filter types), toggle buttons (All/Replies/RTs/Original/High Engagement), paginated result table (25 rows/page), "Tweets by Day" SVG line chart, "Top Handles" SVG bar chart, CSV export via Blob API.
- Added filter bars to all existing pages: handles (search, min followers, cluster, sort), hashtags (search, min uses, topic), topics (search, min tweets, min handles), campaigns (date range, hashtag, min co-occurrences).
- Added .filter-bar, .filter-group, .filter-input, .filter-select, .filter-btn, .filter-tag CSS components to frontend/css/components.css.
- Updated navigation across all 7 pages to include "Tweet Analysis" link. Updated backend/server.py with /tweet-analysis.html route. Updated nav.js router.
- Completed integration testing: verified all API endpoints return correct data, confirmed skeleton loaders display correctly, tested filter combinations on Tweet Analysis page, verified CSV export downloads correctly, confirmed Neo4j health and Postgres health both Online.

WEEKLY PROGRESS REPORT (25/05/2026 – 31/05/2026)

WEEK 2 — MEMBER CONTRIBUTIONS

Rishav Pal — Data Engineer

- Led PostgreSQL integration — designed and built storage/postgres_writer.py with full schema initialisation, upsert logic, and async/thread-safe operation via asyncio.to_thread.
- Built storage/tweet_csv_writer.py — master CSV export with pipe-separated array fields, atomic append writes, QUOTE_ALL encoding, and existing-ID deduplication.
- Implemented cross-run deduplication in scraper/tweet_scraper.py — seen_ids parameter, load_seen_ids_from_jsonl() static method, merged ID sets from JSONL and CSV.
- Updated main.py pipeline to wire PostgreSQL writes and deduplication into the scrape loop.
- Built backend/postgres_client.py — ThreadedConnectionPool, RealDictCursor queries, verify_connection() health check, integrated into /health endpoint.
- Ran storage/backfill_postgres.py to migrate all 1,601 existing tweets into PostgreSQL.
- Created scratch/verify_csv.py — automated validation of CSV deduplication and JSONL-to-CSV consistency.
- Executed full re-scrape test confirming deduplication correctly skips all 1,601 already-scraped tweets on second run.

Sayantan Mandal — OSINT Analyst

- Led frontend design and implementation across all 7 pages of the linkmap dashboard.
- Established the global design system: CSS tokens, typography scale, animation library, component patterns. Implemented the editorial Swiss aesthetic with Playfair Display headings.
- Built frontend/index.html Overview page — animated SVG network preview, live stat counter, project description, and metrics strip fetching from /api/dashboard/stats.
- Built frontend/handles.html, hashtags.html, topics.html — all three pages with animated bar charts, skeleton loaders, paginated tables, and filter bars.
- Built frontend/tweet-analysis.html and tweet-analysis.js — full tweet search interface with all 6 filter types, SVG charts, pagination, and Blob CSV export.
- Added filter bar CSS components to components.css and implemented client-side filtering logic across handles.js, hashtags.js, topics.js, campaigns.js.
- Conducted end-to-end integration testing of all pages against live data from both PostgreSQL and Neo4j backends.
- Verified all OSINT data is correctly displayed: rhythms22 anomaly visible on network page, #JusticeForTwisha burst confirmed on campaigns page, topic clusters match Week 1 findings.

Shivam Kumar — Graph Analyst

- Led Neo4j tweet-free schema migration — designed and implemented the new 4-node schema (User, Hashtag, Link, Topic) with direct entity relationships.
- Rebuilt all Cypher files: load_nodes.cypher, load_relationships.cypher, constraints.cypher, campaign_detection.cypher for the tweet-free schema.
- Added _build_tweet_free_rels() to graph/transformer.py generating three new CSVs: rels_user_hashtag.csv, rels_user_link.csv, rels_user_shared_hashtag.csv.
- Rebuilt backend/routes/dashboard.py, handles.py, hashtags.py, topics.py, campaigns.py to read from PostgreSQL for tweet data while keeping Neo4j for graph relationships.
- Built backend/routes/tweet_analysis.py with 3 endpoints, all using parameterised PostgreSQL queries with full filter support (handle, topic, hashtag, keyword, date range, type flags).
- Built frontend/network.html and frontend/js/network.js — D3.js force-directed graph with Neo4j-style dark canvas, coloured nodes, zoom/pan, Node Inspector panel.
- Built frontend/campaigns.html and frontend/js/campaigns.js — campaign detection page with coordinated activity alert, burst events table, radial hashtag hub SVG.
- Restructured project into backend/ and frontend/ separation, updated all import paths, verified server starts correctly with python main.py --mode frontend.

WEEK 2 — DASHBOARD SCREENSHOTS

The following screenshots show the completed linkmap OSINT Intelligence Dashboard built during Week 2. The dashboard runs at http://localhost:8080 and connects to both PostgreSQL (tweet data) and Neo4j (graph relationships) backends.

Overview Page — Landing Dashboard

The Overview page displays the project introduction, live stat counters (handles, tweets, relationships) fetched from the FastAPI backend, an animated SVG network preview, and the full navigation to all 7 pages.

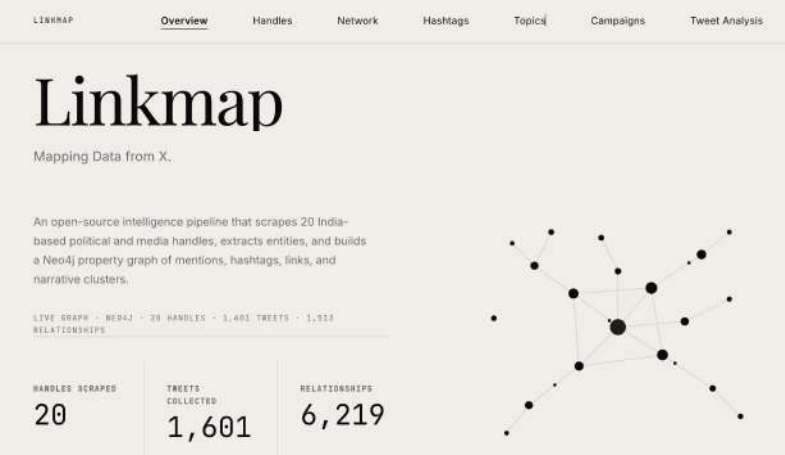

Figure 1: Overview page — live stats and animated network preview

Handle Intelligence Page

The Handles page shows all 20 scraped handles with follower counts, cluster classification, tweet counts, hashtag usage, and mention volume. Includes a filter bar (search, min followers, cluster, sort) and animated horizontal cluster frequency bars.

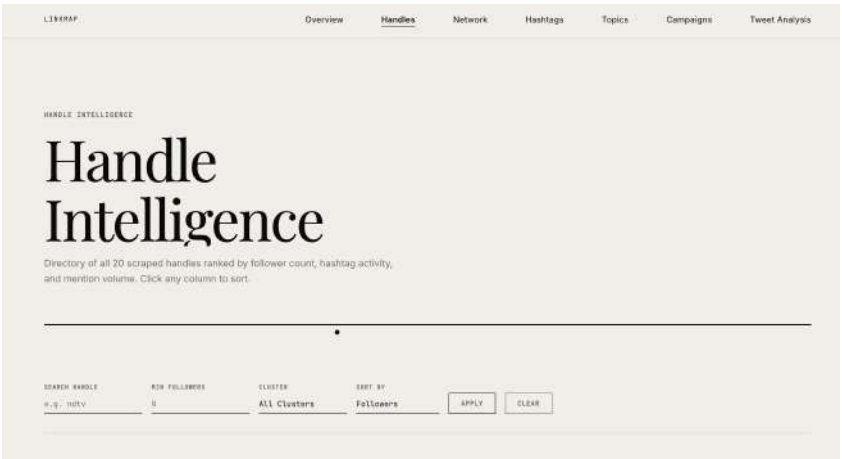

Figure 2: Handle Intelligence page — filter bar and cluster bars

WEEK 2 — DASHBOARD SCREENSHOTS (continued)

Influence Network Page — D3.js Graph

The Network page renders the Neo4j graph data as a D3.js force-directed graph on a dark navy canvas. Node colours match Neo4j Browser style. Click any node to open the Node Inspector panel showing connections and properties.

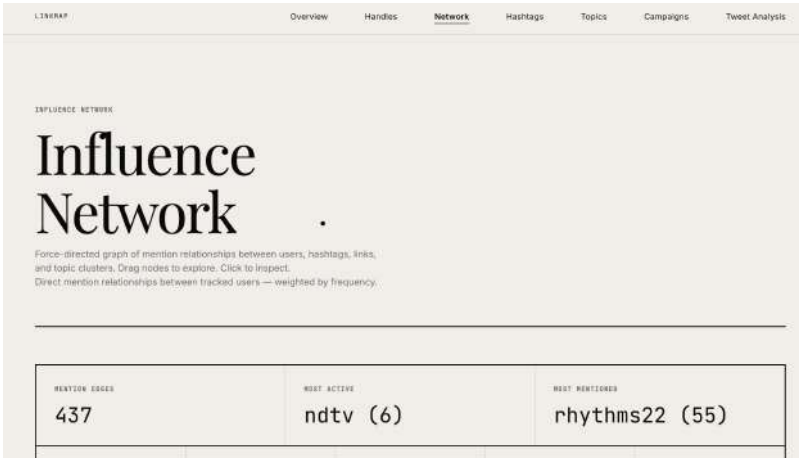

Figure 3: Influence Network — D3.js force graph with Neo4j-style nodes

Hashtag Analysis Page

The Hashtags page shows top-10 hashtag frequency bars (data from PostgreSQL), co-occurrence pairs, and a campaign burst signal. The filter bar allows searching by hashtag name, minimum uses, and topic category.

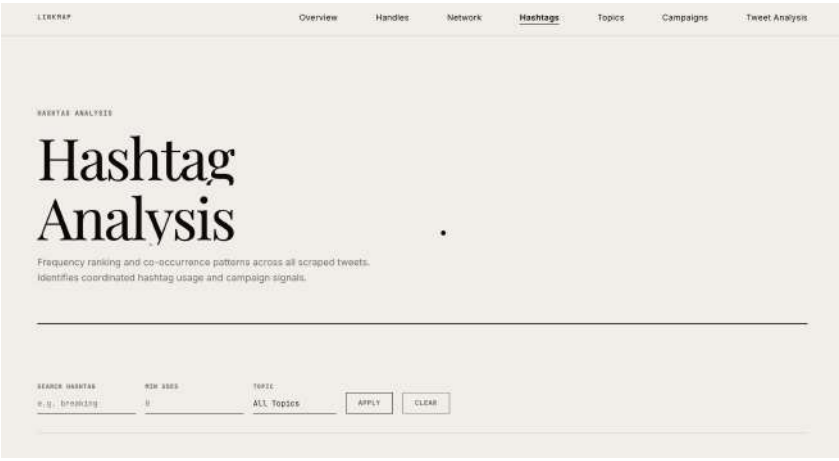

Figure 4: Hashtag Analysis — frequency bars and filter controls

Topic Clusters Page

The Topics page displays all 12 topic clusters with tweet counts, handle counts, and top hashtags. The sidebar shows a vertical distribution chart. Clicking any row expands inline to show handles, hashtags, and links for that topic.

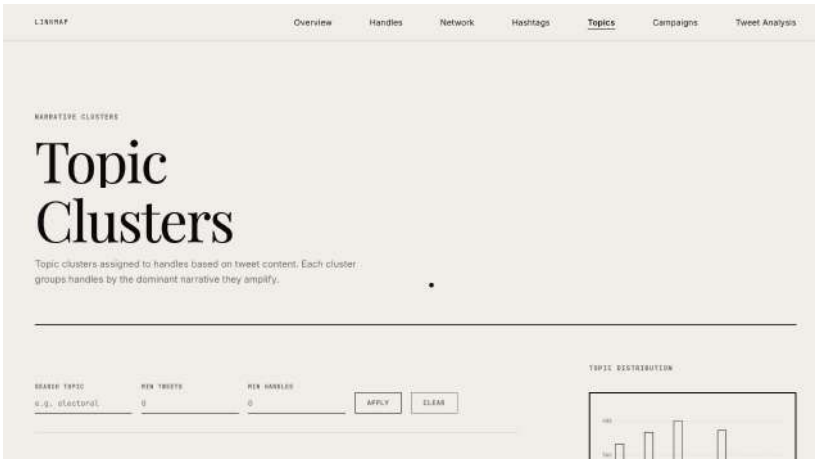

Figure 5: Topic Clusters — distribution chart and filter bar

WEEK 2 — DASHBOARD SCREENSHOTS (continued)

Campaign Detection Page

The Campaigns page shows the coordinated activity alert for Republic TV's #Justice ForTwisha burst campaign detected on 2026-05-22 at 15:00 UTC. The radial hub SVG shows the 7-hashtag cluster and the co-occurrence table confirms 14 tweets in one hour.

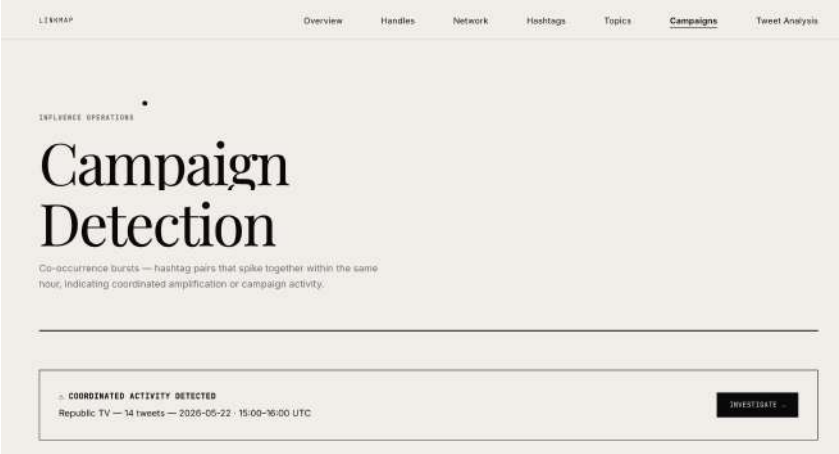

Figure 6: Campaign Detection — coordinated activity alert and investigation

Tweet Analysis Page

The Tweet Analysis page is the full PostgreSQL-powered tweet search interface. It supports 6 simultaneous filters (handle, topic, hashtag, keyword, date range, type), displays matching tweets in a paginated table with "Tweets by Day" and "Top Handles" SVG charts, and exports results as CSV.

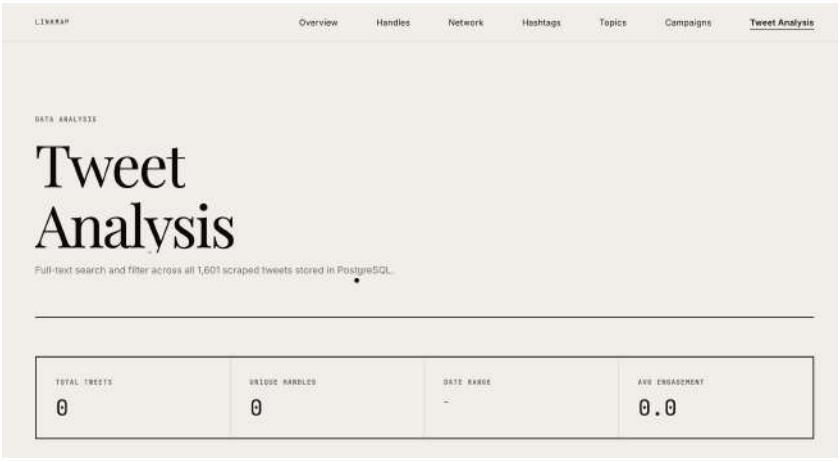

Figure 7: Tweet Analysis — filter bar, stats strip, and data table

Week 2 Technical Summary

Component	Deliverable	Status
PostgreSQL Integration	profiles + tweets tables, upsert, backfill	✓ Complete
Tweet CSV Export	data/tweets/all_tweets.csv, pipe-separated	✓ Complete
Cross-run Deduplication	seen_ids from JSONL + CSV, O(1) skip	✓ Complete
Tweet-free Graph Schema	4 node types, 6 direct relationships	✓ Complete
FastAPI Backend	backend/ folder, 8 route files, security	✓ Complete
Postgres Client	ThreadedConnectionPool, RealDictCursor	✓ Complete

Frontend Dashboard	7 HTML pages, editorial design, D3.js graph	✓ Complete
Tweet Analysis Page	6 filters, SVG charts, CSV export	✓ Complete
Filter Bars	All 6 pages (except index + network)	✓ Complete

Report prepared by: Rishav Pal (Data Engineer), Sayantan Mandal (OSINT Analyst), Shivam Kumar (Graph Analyst) | Week 2: 25 May – 31 May 2026